

## 第四章 软件设计 选择题解析（深度版 v2）

对应 第四章\_软件设计\_选择题\_v2.md（10题全深度版）。每题按首句点破 → 逐选项审判 → 点穿陷阱 → 金句的四段式深度解析。

### 第 1 题

答案：D

解析：

这题考的是内聚 7 级的精确定义——关键在“功能内聚是最高”。

内聚 7 级从低到高排序：偶然 → 逻辑 → 时间 → 过程 → 通信 → 顺序 → 功能。

- 偶然内聚：模块内语句无任何关系，纯粹松散堆砌（最差）
- 逻辑内聚：模块完成逻辑相似的多个功能（按参数决定走哪个分支）
- 时间内聚：模块内的语句需同一时间执行（如初始化）
- 过程内聚：模块按特定顺序完成多个功能（顺序重要）
- 通信内聚：模块内语句共享同一数据（如操作同一数据结构）
- 顺序内聚：模块内语句前一输出是后一输入（流水线）
- 功能内聚：所有元素共同完成单一明确功能（最好）
- A 错：偶然内聚是“无任何关系、松散堆砌”——不是“按参数类型决定执行哪个分支”（后者是逻辑内聚）。
- B 错：逻辑内聚是“按参数选择分支”——不是“按特定顺序完成多个功能”（后者是过程内聚）。
- C 错：时间内聚是“同一时间执行”（如多个初始化）——不是“共享同一数据”（后者是通信内聚）。
- D 正确：功能内聚 = 单一明确功能——内聚的最高境界。

内聚 7 级口诀：“偶逻时过通顺功”（从低到高）。常见错误：把“过程内聚”和“顺序内聚”搞混——过程是“按顺序做不同事”，顺序是“前一个输出是后一个输入”（流水线）。

### 第 2 题

答案：C

解析：

这题考的是耦合 7 级的精确定义——关键在“控制耦合传递标志”。

耦合 7 级从低（好）到高（坏）：非直接 → 数据 → 标记 → 控制 → 外部 → 公共 → 内容。

- 数据耦合：两个模块通过参数传递简单数据（最好）
- 标记耦合：两个模块通过参数传递数据结构（部分共享）
- 控制耦合：传递的标志信息决定后者执行逻辑（控制流）
- 外部耦合：模块共享外部数据格式/协议
- 公共耦合：模块共享公共数据环境（如全局变量）
- 内容耦合：一个模块直接访问另一个模块的内部
- A 错：数据耦合是“传递简单数据”，不是“共享公共数据环境”——后者是公共耦合。
- B 错：标记耦合是“传递数据结构”，不是“传递简单数据”——后者是数据耦合。
- C 正确：控制耦合 = 标志信息决定后者执行逻辑。
- D 错：内容耦合是“直接访问内部”（如修改另一个模块的代码），不是“访问全局数据结构”——后者是公共耦合。



耦合 7 级口诀："非数标控外公内"（从低到高）。常见错误：把"数据耦合"和"标记耦合"搞混——数据=简单数据，标记=数据结构（复合）。

---

### 第 3 题

答案：B

解析：

这题考的是软件设计两阶段的边界——总体设计管"结构"，详细设计管"过程"。

软件设计分两阶段：

- 总体设计（概要设计）：划分模块、确定模块功能、模块调用关系、定义接口
- 详细设计：用伪代码/程序流程图/PDL 详细描述每个模块的算法和数据结构

题干：

- 阶段 P：划分模块、确定功能、调用关系、定义接口 → 总体设计
- 阶段 Q：用伪代码/程序流程图描述每个模块算法 → 详细设计
- A 错：把 P 和 Q 颠倒——P 是总体设计不是详细设计。
- B 正确：P 总体设计、Q 详细设计。
- C 错：两阶段不是同一阶段——总体在前、详细在后，分阶段产出不同文档。
- D 错：两阶段不是同一阶段。

设计两阶段口诀："总体画结构、详细写算法"。总体设计产出"模块结构图"，详细设计产出"每个模块的伪代码/流程图"。

---

### 第 4 题

答案：A

解析：

这题考的是McCabe 环形复杂度  $V(G)$  的计算方法——核心公式有 3 个。

McCabe  $V(G)$  的 3 个等价公式：

- $V(G) = \text{判定语句数} + 1$ （最常用）
- $V(G) = \text{边数} - \text{节点数} + 2$ （基于流图）
- $V(G) = \text{封闭区域数} + 1$ （基于平面图）

题干："函数内有 4 个判定语句"——按公式  $V(G) = 4 + 1 = 5$ 。

- A 正确： $V(G) = \text{判定语句数} + 1 = 4 + 1 = 5$ 。
- B 错： $V(G)$  与入口数量无关——这是误解。
- C 错： $V(G)$  与出口数量无关——也是误解。
- D 错： $V(G)$  与总语句数无关—— $V(G)$  只看"判定/分支"复杂度，与"语句多少"无关。

McCabe  $V(G)$  计算口诀："判定数 + 1"（最常用）。 $V(G)$  度量的是控制流复杂度（分支多少），不是代码长度。 $V(G) > 10$  通常认为模块需要拆分。

---

### 第 5 题

答案：C

解析：

这题考的是内聚类型的代码判断——从代码特征判定内聚级别。

题干代码特征：



- 读取全局数据 X → 共享数据
- 提取订单 a、商品 b、客户 c → 三类数据无共享逻辑
- 按顺序处理 a → b → c → 顺序固定

关键特征：

- 无共享数据（订单 a、商品 b、客户 c 是不同数据）→ 不是通信内聚
- 顺序固定（a → b → c）→ 过程内聚（按特定顺序完成多个功能）
- A 错：偶然内聚是“完全无关系”——但本题有明确顺序。
- B 错：逻辑内聚是“按参数选择分支”——本题不是分支结构。
- C 正确：过程内聚 = 按特定顺序完成多个功能（顺序固定是关键）。
- D 错：通信内聚需要“共享同一数据”——本题三类数据无共享。

过程内聚 vs 通信内聚辨析口诀：“过程讲顺序、通信讲共享数据”。如果同时有顺序和共享数据，按“哪个特征更突出”判断——本题顺序突出（a→b→c）但数据不共享，故是过程内聚。

## 第 6 题

答案：C

解析：

这题考的是设计原理 4 概念的精确辨析——A/B/C/D 各自有微妙错误。

4 个设计原理：

- 抽象：抽出本质特征，暂时忽略细节
- 逐步求精：从顶层开始逐层细化设计
- 模块化：将程序分解为独立模块分别开发
- 信息隐藏：模块只暴露接口，隐藏内部实现
- A 错：抽象是“暂时忽略细节”，不是“逐层细化设计”——后者是逐步求精的核心动作，把“逐层细化”套到抽象上属于张冠李戴。
- B 错：逐步求精是“逐层细化”，不是“暂时忽略细节”——后者是抽象的核心动作，把“暂时忽略”套到逐步求精上。
- C 正确：信息隐藏 = 模块只暴露接口 + 隐藏内部实现。
- D 错：模块化是“分解为独立模块”，不是“抽出本质特征”——后者是抽象的核心动作。

4 原理辨析口诀：“抽象抽本质、细化逐层分、模块分而治、隐藏藏细节”。4 个原理易混点：抽象与信息隐藏（都涉及“隐藏”）、抽象与逐步求精（都涉及“忽略细节”）、抽象与模块化（都涉及“抽取”）——必须知道每个原理的完整定义才能区分。

## 第 7 题

答案：A

解析：

这题考的是 3 种详细设计图形工具的精确辨析——T1/T2/T3 名称。

3 种工具：

- 程序流程图：传统流程图，方框 = 语句、菱形 = 判断、有箭头
- N-S 图（盒图）：用 N-S 盒代替箭头，所有结构在盒内完成（无箭头）
- PAD 图（Problem Analysis Diagram）：用树形结构描述程序逻辑，从左到右展开

题干：

- T1：方框 + 菱形 + 传统流程图 → 程序流程图
- T2：N-S 盒代替箭头，结构在盒内 → N-S 图
- T3：树形结构，从左到右展开 → PAD 图



- A 正确：T1=程序流程图、T2=N-S 图、T3=PAD 图。
- B 错：T1 和 T2 颠倒。
- C 错：T1 和 T3 颠倒。
- D 错：T2 和 T3 颠倒。

3 种工具口诀："方菱箭头=程序流；盒内完成=N-S；树状展开=PAD"。N-S 图的特点是没有箭头（用盒的位置表示顺序），是常考特征。

## 第 8 题

答案：A

解析：

这题考的是面向数据流设计的两种类型——变换型 vs 事务型。

两种类型的判别：

- 变换型：DFD 表现为输入路径 → 变换中心 → 输出路径，单一主线
- 事务型：DFD 表现为事务中心，按输入数据类型分派到不同处理路径

题干："单一处理从输入开始、经过加工后产生单一输出、无明显中心调度节点"——这描述了输入→变换中心→输出的变换型特征。

- A 正确：变换型 = 识别输入路径、变换中心、输出路径。
- B 错：事务型需要"明显的事务中心"——题干说"无明显中心调度节点"，不是事务型。
- C 错：题干说"DFD"是结构化分析输出，面向对象设计不是从 DFD 出发的。
- D 错：过程式设计是"自顶向下逐步细化"是总体策略，不是"面向数据流"的专门方法。

变换型 vs 事务型口诀："变：单入单出有线型；事：多入多出有中心"。没有中心调度节点是变换型的关键特征，不是事务型。

## 第 9 题

答案：A

解析：

这题考的是模块结构的启发式规则——扇出/扇入/深度的合理范围。

启发式规则（McCabe 等教材总结）：

- 扇出（fan-out）：模块调用下层模块数，合理范围 3-7，建议控制在 3-4
- 扇入（fan-in）：模块被上层调用数，越大越好（重用性高），不设上限
- 深度（depth）：模块嵌套层数，应适中（通常不超过 6-7 层）
- 宽度（width）：同一层模块总数，应均匀分布（不出现瓶颈）

题干：扇出 7 → 偏大（应控制在 3-4），应调整。

- A 正确：扇出 7 偏大，应控制在 3-4 范围内。
- B 错：扇入 8 已经很好（重用性高），不是"过小"——扇入越大越好。
- C 错：深度 6 适中，不需要增加到 10 层——深度过深反而难维护。
- D 错：扇出扇入深度都有明确规则，不是"无规则"。

启发式规则口诀："扇出 3-4 不超 7；扇入越大越好；深度不超 6-7"。扇出过大（>7）= 模块太复杂，应拆分；扇入过小（<3）= 模块未被重用，应考虑是否需要。

## 第 10 题



答案：B

解析：

这题考的是设计原理的场景识别——模块 M 的具体行为对应哪个原理。

题干场景：模块 M 公开 process()（外部只能通过它调用）+ 内部使用数据结构 X 和算法 Y（外部无法访问 X、不能直接调用 Y）。

这正是信息隐藏的典型场景：

- 公开接口（process()）
- 隐藏内部实现（X 和 Y 都不可见）
- 外部只能通过接口与模块交互
- A 错：抽象是"暂时忽略细节"——本题是"永久封装"，是信息隐藏不是抽象。
- B 正确：信息隐藏 = 模块只暴露接口 + 隐藏内部实现（X 和 Y 都不可见）。
- C 错：模块化是"分解为独立模块"——本题场景是单个模块的内部设计原则，不是模块化。
- D 错：逐步求精是"逐层细化"——本题是"封装内部"，不是细化过程。

信息隐藏 vs 抽象辨析口诀："信息隐藏=永久封装（接口外不可见）；抽象=暂时忽略（设计阶段忽略）"。本题的"process() 公开 + X/Y 隐藏"是信息隐藏的标志场景。

## 答案速查

| 题号 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|----|---|---|---|---|---|---|---|---|---|----|
| 答案 | D | C | B | A | C | C | A | A | A | B  |